

Multi-vehicle Chassis Control

Multi-vehicle Chassis Control

1. Content Description
2. Preparation Work
 - 2.1 Basic Requirements
 - 2.2 Setting Vehicle Unit `DOMAIN_ID`
 - 2.2.1 Setting Vehicle Unit Main Control `DOMAIN_ID`
 - 2.2.2 Setting Vehicle Unit STM32 Main Control Board `DOMAIN_ID`
 - 2.3 Modify Vehicle Unit Namespace
3. Starting Example Programs
 - 3.1 Node Communication
 - 3.2 Principle Analysis

1. Content Description

- Run example programs to implement simultaneous control of multiple vehicle units using the keyboard. This tutorial uses 2 vehicle units as an example.

2. Preparation Work

2.1 Basic Requirements

Taking two vehicles as an example, these two vehicles need to simultaneously meet the following three requirements:

- **Both vehicle units need to be on the same local area network**
- **The `ROS_DOMAIN_ID` of the main control ROS environment for both vehicle units needs to be the same**
- **The `namespace` of both vehicle units must be set to different values**

2.2 Setting Vehicle Unit `DOMAIN_ID`

- The factory default is 30. If you need to modify it to a specified value, refer to this part of the tutorial. Normally, you don't need to modify it as long as you ensure that the multi-vehicle `DOMAIN_ID` is the same.

2.2.1 Setting Vehicle Unit Main Control `DOMAIN_ID`

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

- Enter the following command in the terminal to edit the environment file:

```
nano .bashrc
```

Find `ROS_DOMAIN_ID`, modify it to the same value, then save and exit

```
GNU nano 6.2 .bashrc
source /opt/ros/humble/setup.bash
source /home/jetson/mircoROS_agent/install/setup.bash
#Code space environment variables
source /home/jetson/yahboomM3_ws/code_ws/install/setup.bash
#Code space environment variables
source /home/jetson/yahboomM3_ws/user_ws/install/setup.bash
#ROSMaster-M3 Code space environment variables
export PYTHONPATH="/home/jetson/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
export ROS_DOMAIN_ID=30
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export ROBOT_TYPE="ROSMaster-M3"

#=====雷达和相机型号=====
#=====Radar and camera models=====
#=====
#单雷达: single          双雷达: dual
#Single radar: single    Dual radar: dual
export LASER_TYPE=single
#export LASER_TYPE=dual
#=====
#相机类型    dabai_dcw2          as_hp60c
#Camera type dabai_dcw2          as_hp60c
```

- Refresh the environment variables in the terminal to see the updated `DOMAIN_ID`

```
jetson@yahboom: ~
[System Information]
-----
IP_Address_1: 192.168.2.84
IP_Address_2: 192.168.12.56
MACHINE: OrinNano | ROS_DOMAIN_ID: 30 | ROS_DISTRO: humble
ROBOT_TYPE: ROSMASTER-M3 | LASER_TYPE: single | CAMERA_TYPE: dabai_dcw2
-----
jetson@yahboom:~$
```

2.2.2 Setting Vehicle Unit STM32 Main Control Board DOMAIN_ID

- Edit the configuration program file (any editing method can be used, nano is used here as an example)
- Enter in the host machine terminal,

```
nano ~/config_robot.py
```

Modify the value in `robot.set_ros_domain_id(30)`, then press `ctrl+x` to save

```
jetson@yahboom: ~
GNU nano 6.2 /home/jetson/config_robot.py
print("-----Open %s-----" % com)
else:
    robot = MicroROS_Robot(port="/dev/myserial", debug=debug)

print("Waiting to read the car type")
while True:
    car_type = robot.read_car_type()
    if car_type is not None:
        print("car_type:", car_type)
        if int(car_type) == robot.CAR_TYPE:
            break
    time.sleep(.5)

robot.set_ros_domain_id(30)
#robot.set_ros_namespace("")
# robot.set_motor_pid_parm(0.8, 0.06, 0.5)
# robot.set_imu_yaw_pid_parm(0.6, 0, 0.3)

# robot.set_ros_scale_line(1.0)
# robot.set_ros_scale_angular(1.0)

# arm_mid_value = [2000, 2000, 2000, 2000, 1486, 3100]
# robot.set_arm_mid_value(arm_mid_value)
```

- If the MicroROS agent is started, please close the agent first, then run the configuration program to write the DOMAIN_ID to the chassis control board

```
python3 config_robot.py
```

```
jetson@yahboom: ~
jetson@yahboom:~$ python3 config_robot.py
Read device: Linux
Waiting to read the car type
car_type: 7
version: 1.2.0
domain_id: 30
ros_namespace:
ros_scale_line: 1.000
ros_scale_angular: 1.000
motor pid parm: 0.800, 0.060, 0.500
imu pid parm: 0.600, 0.000, 0.300
arm_mid: 1984,2052,1982,2068,1486,2972
jetson@yahboom:~$
```

2.3 Modify Vehicle Unit Namespace

- The `namespace` of both small vehicles must be set to different values to prevent underlying data interference.
- Set the `namespace` of the first vehicle unit to `robot1`, and other vehicle units follow this pattern

```

# print("state=", state)

# angle= bot.get_uart_servo_angle(1)
# print("angle:", angle)

# angle_array = bot.get_uart_servo_angle_array()
# print("angle_array:", angle_array)

# bot.set_car_motion(0.5, 0, 0)
# time.sleep(1)
# bot.set_car_motion(0, 0, 0)

bot.set_ros_domain_id(30)
bot.set_ros_namespace('robot1')
#bot.set_uart_servo_angle_array()
#bot.set_uart_servo_torque(0)

#state = bot.set_uart_servo_offset(2)
#print("state=", state)

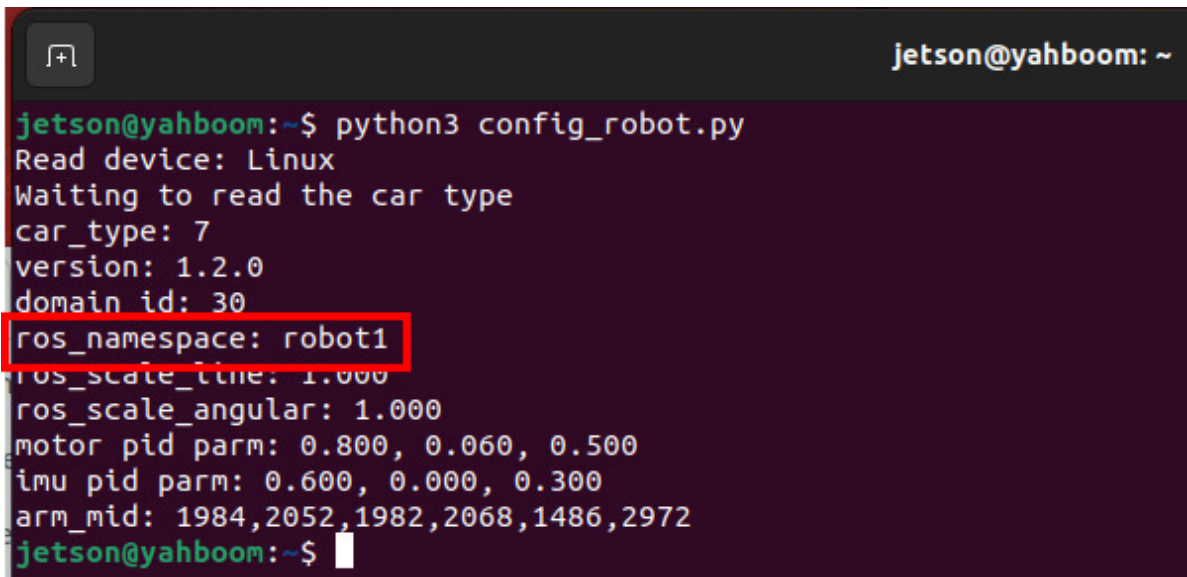
#state = bot.set_uart_servo_offset(3)
#print("state=", state)

#state = bot.set_uart_servo_offset(4)
#print("state=", state)

```

- If the MicroROS agent is started, please close the agent first, then run the configuration program to write the `namespace` to the chassis control board

```
python3 config_robot.py
```



```

jetson@yahboom: ~$ python3 config_robot.py
Read device: Linux
Waiting to read the car type
car_type: 7
version: 1.2.0
domain id: 30
ros_namespace: robot1
ros_scale_tire: 1.000
ros_scale_angular: 1.000
motor pid parm: 0.800, 0.060, 0.500
imu pid parm: 0.600, 0.000, 0.300
arm_mid: 1984,2052,1982,2068,1486,2972
jetson@yahboom: ~$

```

3. Starting Example Programs

- Start the chassis communication for all vehicle units, enter the command in the vehicle unit terminal:

```
start_agent
```

The terminal prints the following information, indicating successful connection

```
mircoROS_Agent
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1765183764.822476] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1765183764.826313] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x006(2), participant_
id: 0x000(1)
[1765183764.829482] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1765183764.834154] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1765183764.838233] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x10D9EFBC, topic_id: 0x007(2), participant_
id: 0x000(1)
[1765183764.842504] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x10D9EFBC, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1765183764.847468] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x10D9EFBC, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

After completing the namespace setting and **reconnecting** the chassis agent, enter the following command in the corresponding terminal to check whether the `namespace` and `ROS_DOMAIN_ID` are set correctly

```
ros2 node list
```

As shown in the figure below, the appearance of `/robot1/YB_Node` and `/robot2/YB_Node` indicates successful setting,

```
yahboom@yahboom-VM:~$ ros2 node list
/robot1/YB_Node
/robot2/YB_Node
```

- Enter command in terminal

```
keyboard_control
```

After the program starts, click on the terminal that started keyboard control, and press the corresponding keys according to the following key table to control the movement of two vehicles,

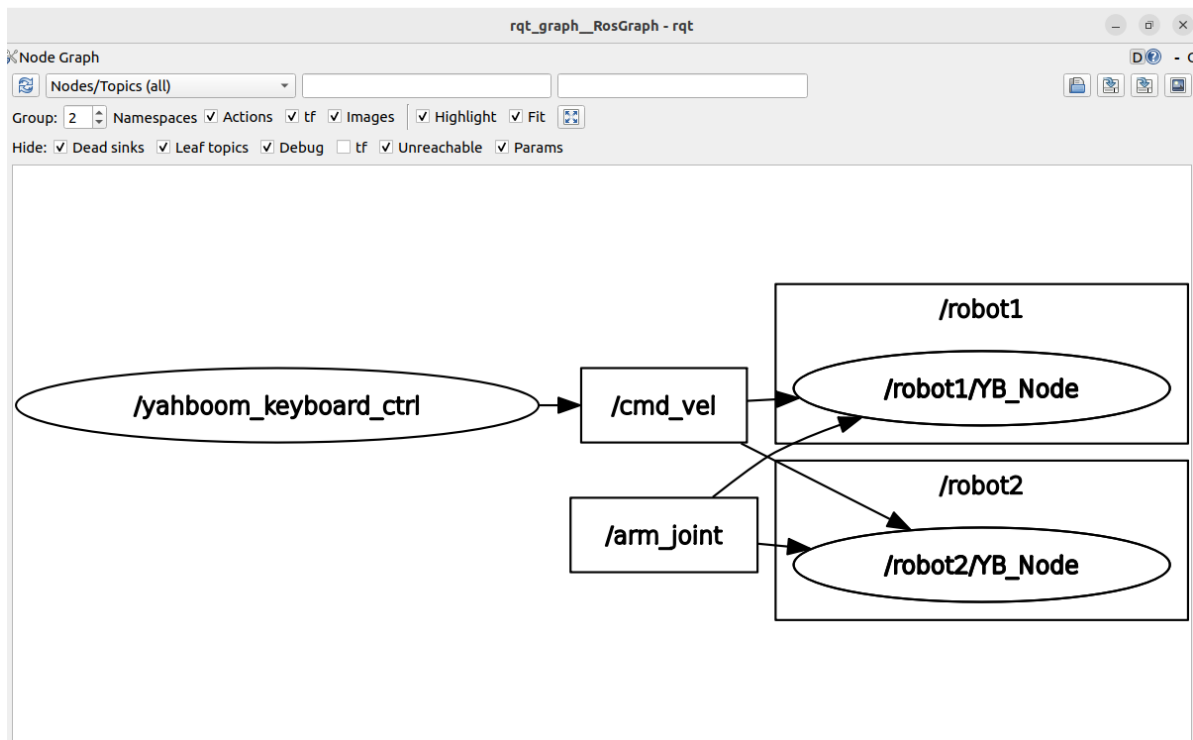
Key	Action
i or I	Forward
<	Backward
j or J	Turn left in place
l or L	Turn right in place
u or U	Forward left turn
o or O	Forward right turn
m or M	Backward left turn
>	Backward right turn
k	Stop

3.1 Node Communication

Enter the following command in the virtual machine terminal to view the node communication diagram,

```
ros2 run rqt_graph rqt_graph
```

As shown in the figure below, select [Nodes/Topics(all)] in the upper left corner, then click the refresh button on the left



3.2 Principle Analysis

The keyboard control node `/teleop_twist_keyboard` publishes the speed topic `/cmd_vel`. The underlying nodes of both vehicles `/robot1/YB_Node` and `/robot2/YB_Node` subscribe to this `/cmd_vel` topic. After receiving the topic message data, they process it and pass it to the driver board, thereby controlling the vehicle movement.